

MaiAgent Technical Documentation

Table of Contents

1. Generative AI Quickstart

- 1.1 Large Language Models (LLM)
- 1.2 RAG Knowledge Base Retrieval System
- 1.3 Embedding Models
- 1.4 Reranker Models
- 1.5 Parser Tools
- 1.6 Image Recognition Support

2. MaiAgent Work | Employee Productivity Integration

- 2.1 Add MaiGPT to Your System
- 2.2 Integrate MaiGPT: Choose the Right Assistant and Add the Capabilities You Need
- 2.3 Member Memory Management API

3. MaiAgent Serve | Customer Service Integration

- 3.1 Software Vendor Integration Guide: Embed AI Agents for Your Customers
- 3.2 Web Chat Embedding and SDK
- 3.3 Embedding in MaiGPT Mode
- 3.4 Inject Page Context
- 3.5 LINE LIFF: What Is LINE LIFF?
- 3.6 LINE LIFF: How to Integrate
- 3.7 Introduction to Contacts and Integration
- 3.8 Contact Identity Synchronization and Token Updates
- 3.9 Differences Between Roles and Contacts
- 3.10 Third-Party Customer Service System Integration (Including Human Agent Handoff)
- 3.11 Conversation Visibility Permissions and Real-Time Events
- 3.12 Conversation Handoff and Continuation API
- 3.13 Webhook

4. MaiAgent Build | AI Assistant Modules

- 4.1 System Prompts
- 4.2 Knowledge Base
- 4.3 FAQ Management
- 4.4 Response Evaluation and Monitoring Results
- 4.5 AWS Guardrails

5. MaiAgent Build | Advanced Generative AI Technology

- 5.1 Text to SQL
- 5.2 Function Calling
- 5.3 Skill Modules
- 5.4 Canvas
- 5.5 AI Security Safeguards
- 5.6 MCP (Model Context Protocol)
- 5.7 OAuth 2.0 Integration and Automatic Token Refresh
- 5.8 ICAP Content Validation Integration

6. MaiAgent Build | API Reference

- 6.1 Quickstart
- 6.2 AI Assistant List
- 6.3 Conversations and Message Responses (Streaming/Synchronous)
- 6.4 Create Conversations and Messages
- 6.5 Presigned File Upload Mode
- 6.6 Conversation Attachment Management API
- 6.7 Conversation Output List API
- 6.8 Using the Knowledge Base—Creation and Interaction
- 6.9 API Tool Engineering Guide
- 6.10 Self-Hosted MCP Server Integration Specification
- 6.11 Site Branding Settings API
- 6.12 Credits Item Display Order
- 6.13 Retirement of the Built-In Workflow Reply Mode

7. MaiAgent Build | AI Gateway

- 7.1 AI Gateway: OpenAI-Compatible API Integration
- 7.2 AI Gateway Security

8. MaiAgent Build | External Service Integration

- 8.1 Remote MCP Service Overview
- 8.2 Composio Integration
- 8.3 Zapier Integration
- 8.4 Google Sheet Integration
- 8.5 n8n Integration

9. Identity and Access

- 9.1 Member Provisioning: SCIM/Invitations/SSO
- 9.2 Real-Time Synchronization: Calls from User Events
- 9.3 Roster Synchronization Tool: CSV/JSON/Database
- 9.4 SAML SSO Integration
- 9.5 OAuth Refresh Token Revocation and Rotation
- 9.6 Knowledge Management Permissions (Query Metadata) Overview
- 9.7 Start Building—Using the Query Builder
- 9.8 Start Building—Using JSON

10. Platform Setup and Deployment

- 10.1 Overview
- 10.2 Platform Environments (SaaS/Private Cloud/On-Premises)
- 10.3 Deployment Architecture
- 10.4 Cloud Model Inference API Service
- 10.5 GPU Compute Hardware Planning
- 10.6 Celery Periodic Task Configuration and OAuth Token Refresh
- 10.7 Online Office File Editing in the File Library (ONLYOFFICE Docs Integration)

11. System Updates

- 11.1 Knowledge Base File Creation and Metadata Processing Improvements

Generative AI Quickstart

Large Language Models (LLM)

Key Selection Considerations

When selecting a large language model, consider the following key factors:

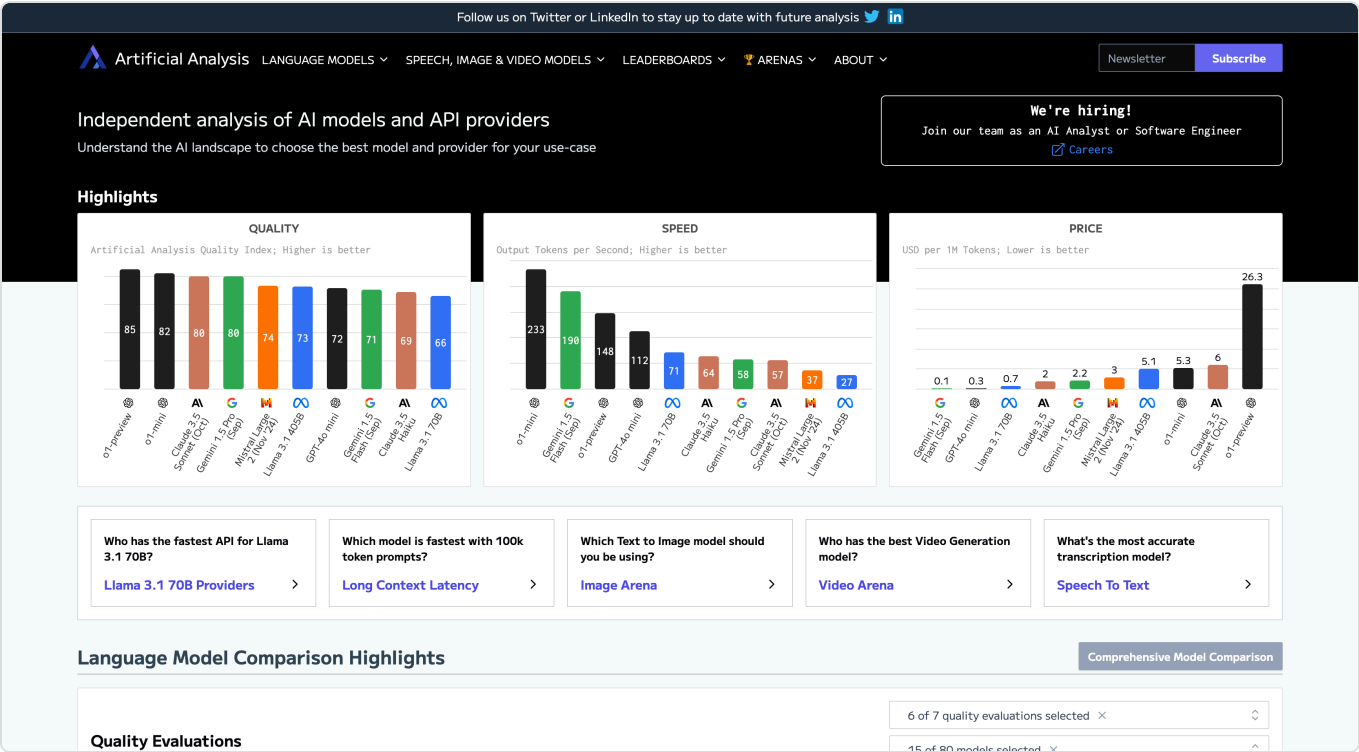
Environment: Determine whether to use cloud-based or on-premise models based on internet connectivity.

Quality: The model's ability to generate responses and adherence to instructions.

Speed: Text generation speed and latency requirements to ensure timely model responses.

Pricing: Consider the model's usage cost and select an appropriate model based on usage requirements. (No need to consider model pricing on MaiAgent)

Other: Whether it supports multimodality and function calling.



Large Language Model Analysis - [Artificial Analysis](#)

Large Language Models Supported on MaiAgent

Closed-Source Models

Model Name	Description	Is Reasoning Model	Use Cases
o4-mini	Faster than o3-mini-high, slightly lower quality than o3-mini-high	Yes	High-quality, fast option
o3-mini-high	High quality, medium speed, uses chain-of-thought reasoning for multi-layer computation before answering to provide more complete and accurate responses	Yes	High-difficulty tasks requiring deep reasoning and creativity
o3-mini-medium	Fast speed, medium quality	Yes	Most business applications, simple creative tasks, or regular Q&A
o3-mini-low	Fastest speed , basic quality, lacks deep reasoning	Yes	Simple tasks prioritizing speed over depth
o1-mini 2024-09-12	o1 series large language models are trained with reinforcement learning to perform complex reasoning. o1 models think before answering, generating a long internal chain of thought before responding to users. Slowest speed, good quality .	Yes	Extremely difficult problems when other LLMs fail
GPT-4o 2024-08-06	Above-average quality and speed	No	Instruction following and logic slightly inferior to Claude 3.5 Sonnet, but faster than Claude 3.5 Sonnet. A commonly used choice 👍
GPT-4o mini 2024-07-18	Fast speed, medium quality. Quality slightly lower than Gemini 2.0 Flash	No	Simple tasks, alternative when Gemini 2.0 Flash is unavailable
Claude 4 Sonnet	Medium-to-slow speed, strong structured data generation/extraction capabilities , particularly excels at Tool Calling ; logical reasoning and coding performance surpass Claude 3.7 Sonnet, with further reduced hallucination rate.	Hybrid reasoning model	First choice for Agent mode 👍 Suitable for highly complex tasks, professional domain applications, and extended conversations.
Claude 3.7 Sonnet	Medium-to-slow speed, excels at generating structured data, has stronger logical reasoning than Claude 3.5 Sonnet. Low hallucination rate	Hybrid reasoning model	First choice for most situations 👍 Suitable for high-complexity tasks, professional domains, and long conversations

Model Name	Description	Is Reasoning Model	Use Cases
Claude 3.5 Sonnet	Follows role instructions well, weaker logical reasoning compared to Claude 3.7 Sonnet, but faster speed . Low hallucination rate	No	Can switch to Gemini 2.0 Flash when speed feels too slow
Gemini 2.5 Pro	Better quality than Claude 3.7 Sonnet for longer conversations and code generation, but slightly inferior in Agent mode and tool calling	No	Can be used interchangeably with Claude 3.7 Sonnet
Gemini 2.0 Pro	Similar quality to Claude 3.5 Sonnet, but slower speed	No	Alternative to Claude 3.5 Sonnet
Gemini 2.5 Flash	Fast speed , good multimodal capabilities	No	
Gemini 2.0 Flash	Fast speed , medium quality	No	First choice for simple tasks 👍
DeepSeek V3	Fast speed, high quality	Yes	Suitable for document retrieval and large-scale database query tasks
DeepSeek R1 Distill Llama 70B	High response quality, medium speed (slower than DeepSeek V3)	Yes	Suitable for tasks requiring multi-step reasoning and background knowledge
DeepSeek R1	Slower response speed, but strong Chinese language understanding , high-quality response content . Deep thinking and truly adjusts based on role instruction content	Yes	Complex multi-turn Chinese conversations. Handling complex role instructions 👍

Open-Source Models

Below is a comparison table of mainstream open-source models. For hardware requirements, refer to the [GPU](#) section.

Model Name	Description	Agent Support	Use Cases
GPT-OSS-120B	Excels at organizing information in table format	Yes	Data analysis, content creation
Gemma3 27B	Good image OCR performance	No	Invoice recognition, image analysis

Model Name	Description	Agent Support	Use Cases
Meta Llama3.3 70B	Highly cost-effective general-purpose model with strong instruction following	No	Voice customer service, RAG Q&A assistant
Meta Llama3.2 90B	Vision capabilities, can process both images and text	No	Professional domain Q&A, high-precision tasks
Meta Llama3.1 405B	Extremely broad knowledge and high reasoning ability	Yes	Customer service Q&A, knowledge Q&A
Llama3 Taiwan 70B	Fine-tuned for Traditional Chinese and Taiwanese culture, accurate localized terminology	No	Customer service Q&A, RAG Q&A assistant
DeepSeek R1	Reasoning-enhanced model, excels at solving complex problems through chain of thought	No	Scientific logical reasoning
DeepSeek V3.2	MoE architecture, fast inference speed and extremely low cost	No	Large-scale text summarization
Qwen3 235B	Code and math specialized, deep logical understanding, suitable for hardcore tasks	Yes	Academic paper assistance
Qwen3 32B	Dynamic balance between high-quality answers and efficient processing	Yes	Multilingual dialogue and instruction following
Qwen3 8B	Medium quality, balanced performance and cost	Yes	Chat assistant, customer service FAQ
Qwen2.5 VL 72B instruct	Vision-language model, accurate image detail recognition	Yes	Multimodal chat assistant
Mistral Large (24.07)	Medium quality, lacks deep reasoning ability, fast speed	No	Customer service Q&A, simple text generation

Is Fine-tuning Always Necessary?

With the rapid development of artificial intelligence technology, language models have acquired powerful language understanding and generation capabilities and are widely applied in many fields. For example, pre-trained language models can easily handle daily conversations, article generation, and simple Q&A tasks.

However, when models face more challenging professional domain tasks, such as healthcare, legal, or technical support, relying solely on pre-trained models may not provide optimal performance. Some

developers choose to perform **Fine-tuning**, which involves additional training for specific domains to enhance the model's professional knowledge.

However, Fine-tuning is not the only solution. There are two other effective methods to achieve this goal: **Prompt Engineering** and **RAG (Retrieval-Augmented Generation)**.

1. Prompt Engineering: Optimizing Model Performance Through Precise Prompts

Prompt Engineering involves designing precise prompt statements to guide the model in generating desired results. The core of this approach is to design detailed expressions based on task requirements, helping the model narrow the scope of answers and understand the context and required output format.

Suppose there is a language model whose purpose is to recommend suitable products based on user needs. In this case, if expressed as "I want to buy a high-performance phone," it may lead the model to generate insufficiently precise answers because "high-performance" can have many different interpretations, possibly referring to processing speed, camera performance, battery life, etc.

To improve recommendation accuracy, we can perform **Prompt Engineering** and design more specific questions or provide additional context to guide the model to better understand user needs. For example, change to the following prompt statement:

"I need a phone with **long battery life** and a **powerful processor**, priced between **\$500 and \$800**. Please recommend several phones that meet these criteria."

2. RAG: Combining External Knowledge to Enhance Generation Capabilities

RAG enhances model performance by combining external knowledge retrieval with the generation process.

In traditional generation tasks, models rely only on knowledge learned during pre-training. RAG uses a retrieval system to obtain external data in real-time and combines this data with the generation model to answer questions or generate text more accurately.

For example, in healthcare, when a model is asked about a rare disease, RAG can first retrieve relevant information from professional medical databases, then generate more accurate answers based on this information. The advantage of this method is that even if the model hasn't encountered certain data during training, it can still make high-quality responses by retrieving existing knowledge. This is particularly suitable for scenarios requiring real-time knowledge updates and can greatly expand the model's knowledge scope.

For a more detailed introduction to RAG, see the next chapter "[RAG Knowledge Base Retrieval System Explanation](#)".

In summary, Fine-tuning, Prompt Engineering, and RAG each have their advantages and applicable scopes. You can choose the most suitable strategy based on application scenarios and needs, without relying solely on Fine-tuning.

Prompt Engineering provides a low-cost and flexible solution by designing precise prompts to guide the model to produce high-quality results.

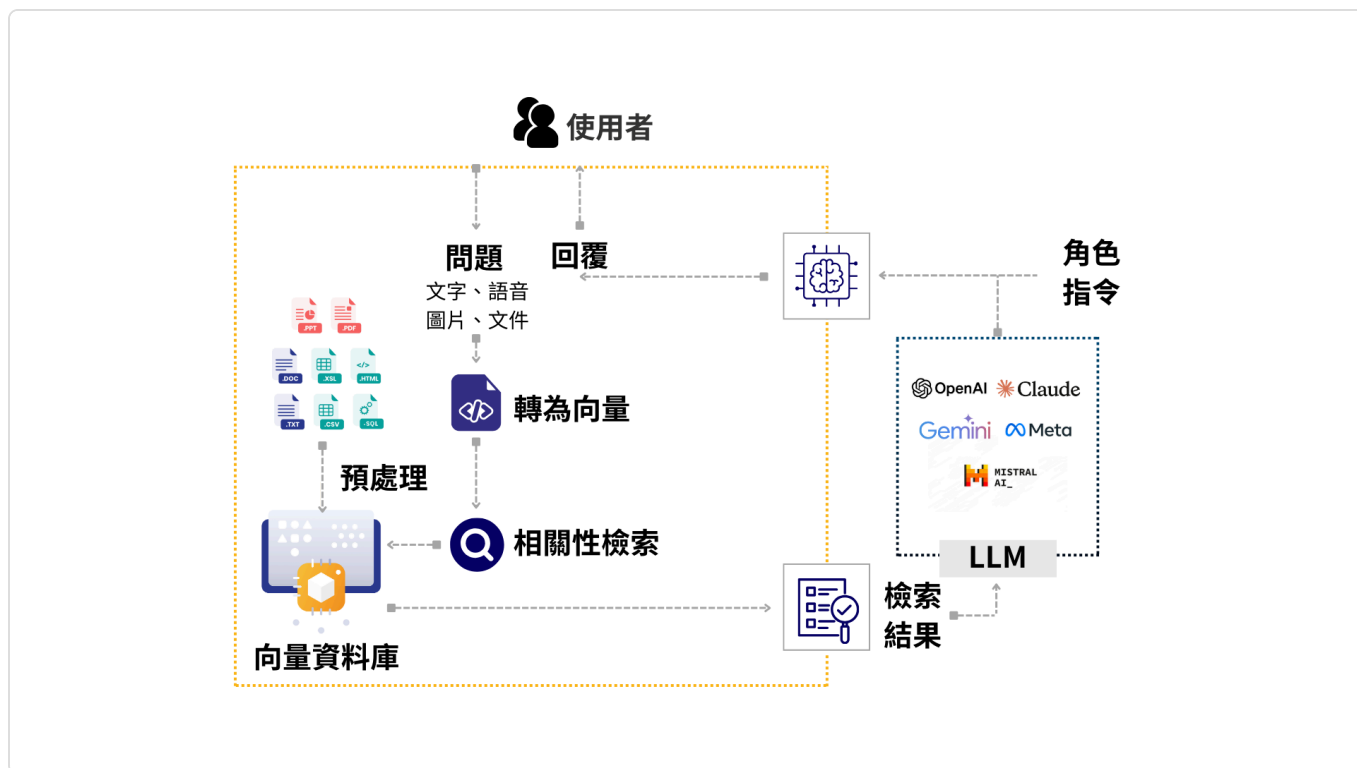
RAG provides a method combining external knowledge with generation capabilities, offering more accurate answers when dynamic knowledge retrieval is needed.

Fine-tuning can significantly improve model performance in specific domains but requires substantial specialized data and consumes additional resources. It should be considered a last resort when both **Prompt Engineering** and **RAG** fail to achieve success.

RAG Knowledge Base Retrieval System

It retrieves information from an external database or knowledge base and combines the results with a large language model to generate more accurate and contextually relevant responses.

At its core, RAG combines the language capabilities of generative AI with knowledge retrieval. When answering questions, the model does not rely solely on its internal training data. It can also dynamically retrieve the latest, more specialized information from external databases and incorporate it into generated responses.



RAG workflow

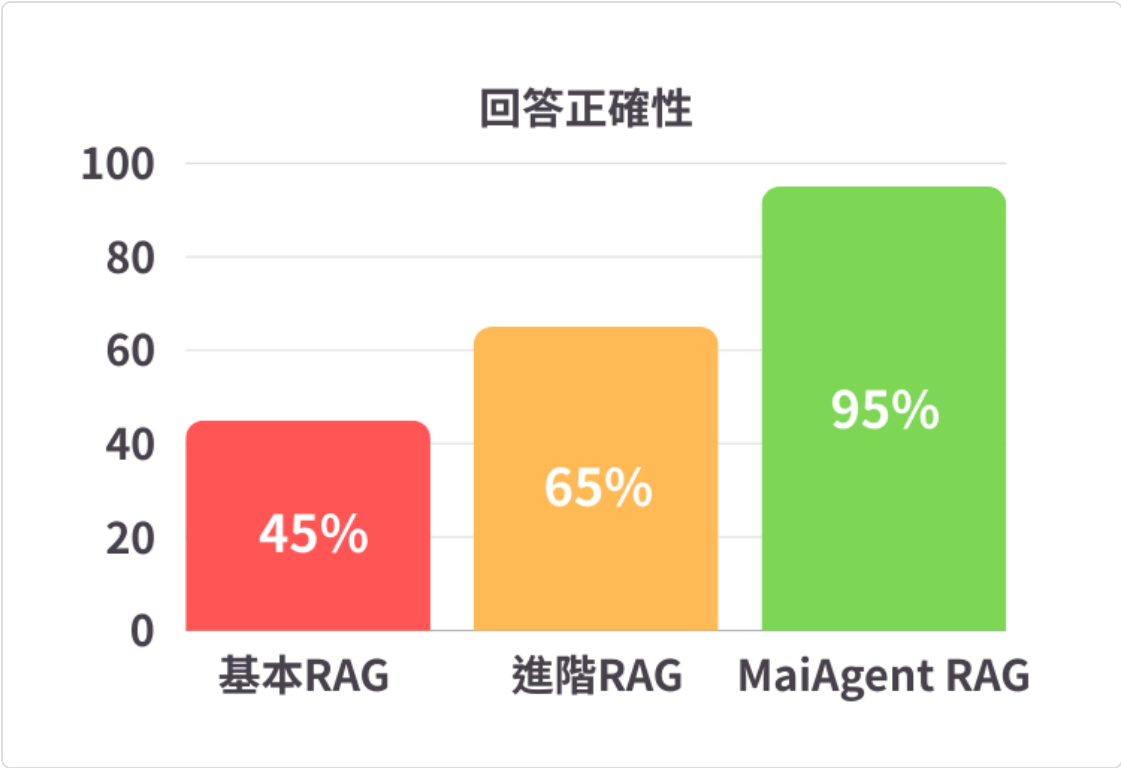
High-Accuracy RAG Systems

Although a basic RAG knowledge base retrieval system can be quickly implemented with Vector Search, further improving response accuracy is challenging. Response accuracy is critical to the user experience because it directly affects users' trust in and satisfaction with the system's responses. If response accuracy is insufficient, users may question the system's answers and become less willing to use it.

According to information presented at OpenAI DevDay 2023, a RAG system that uses only basic Vector Search with the correct Embedding Model can achieve 45% response accuracy. Adding HyDE Retrieval,

FT Embeddings, and Chunk/Embedding Experiments can increase response accuracy to 65%.

In addition to the RAG techniques presented at OpenAI DevDay, MaiAgent RAG combines various classic NLP algorithms with proprietary retrieval technologies. On internal datasets, both MaiAgent RAG and OpenAI RAG achieve 95% response accuracy.



MaiAgent RAG response accuracy

The MaiAgent platform provides two RAG options: MaiAgent RAG and OpenAI RAG. The following table compares them across several dimensions:

	MaiAgent RAG	OpenAI RAG
Model support	Supports all models 👍	Supports only OpenAI models
Environment support	Supports cloud and on-premises environments 👍	Supports only the cloud; data must be sent to OpenAI
Response accuracy	Extremely high 👍	Extremely high 👍
Supported file formats	Supports all common formats 👍 doc, docx, xls, xlsx, csv, ppt, pptx, pdf, txt, json, jsonl, md	Does not support xlsx or csv Does not support jsonl Does not support legacy Office files (doc, xlsx, ppt)
Images in documents	Supported (currently experimental) 👍	Not supported

	MaiAgent RAG	OpenAI RAG
Tables in documents	Supported (currently experimental) 👍	Not supported
Attachment uploads in conversations	Supported 👍	Supported 👍
Data chunking transparency	Visualized 👍	Black box
Debugging difficulty	Moderate 👍	Black box; cannot be debugged
Top K adjustment	Enterprise customization feature 👍	Not supported
Switching Embedding Models	Enterprise customization feature 👍	Not supported

Embedding Models

What Is Embedding?

In the RAG (Retrieval-Augmented Generation) process, the core task of **Embedding** is to convert large volumes of text processed by the Parser, such as documents and knowledge base content, into numerical representations that computers can understand and compare—namely, vectors.

This conversion enables a RAG system to:

1. Understand Semantics

Embedding models capture the deeper semantic meaning of text rather than just surface-level words. This means that even if a query and a document use different wording, the system can recognize their relationship as long as their meanings are similar.

2. Retrieve Efficiently

After converting text into vectors, the system can use efficient vector similarity search algorithms to quickly find the text passages most relevant to a user's query in a large database.

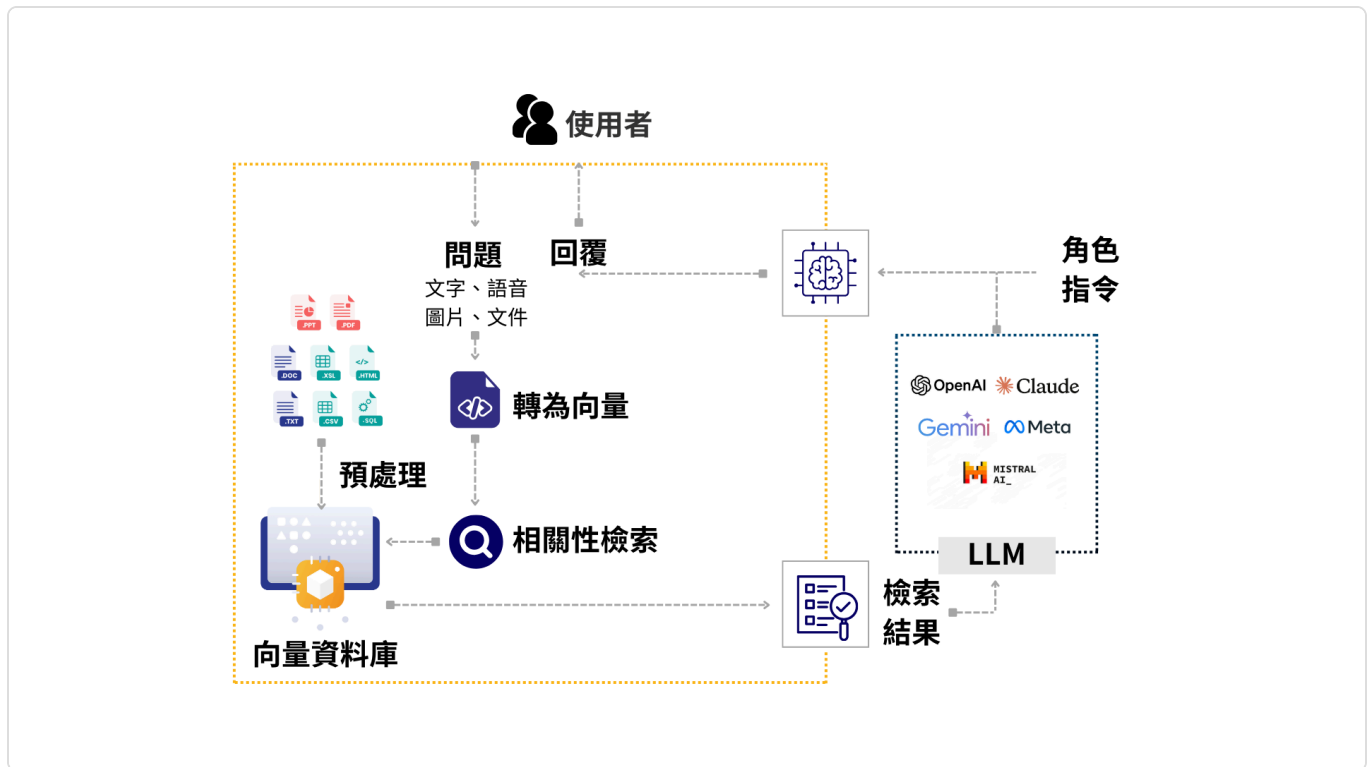
3. Improve Generation Quality

The retrieved text passages are provided to a large language model (LLM) as context, helping the LLM generate more accurate answers.

Embedding Plays Two Key Roles in MaiAgent's RAG Technology

1. Converts knowledge base content into vectors and stores them in a vector database

2. Vectorizes user questions, compares them with vectors in the database, and identifies the most relevant content



RAG workflow

Simply put, Embedding is the foundation of a RAG system. It converts unstructured text into computable and comparable vectors, making it a critical prerequisite for accurate information retrieval and high-quality content generation.

How Embedding Affects RAG Systems

1. Retrieval Quality

Depth of semantic understanding: High-quality Embedding models capture the semantic meaning of text more accurately, improving retrieval relevance

Context awareness: Strong Embedding models understand contextual relationships within text, ensuring coherent retrieval results

Multilingual support: Powerful multilingual Embedding models can handle cross-language knowledge retrieval requirements

2. System Performance

Retrieval speed: The vector dimensions and computational efficiency of an Embedding model directly affect retrieval response time

Resource consumption: Different Embedding models have different computing resource requirements, affecting system scalability

Parallel processing: Efficient Embedding models support large-scale parallel retrieval

Embedding Models Supported by MaiAgent

Model	Developer	Country of Origin	Characteristics	Open Source	Deployment Method	Average MTEB Score
Cohere Embed v4.0	Cohere	Canada	Multilingual support and highest performance	No	Requires a cloud API inference service	Not yet published; v3.0 score of 64.47 can be used as a reference
Cohere Embed Multilingual v3.0 (Bedrock)	Cohere	Canada	Multilingual support and high performance	No	Requires a cloud API inference service	64.47
OpenAI text-embedding-3-Large	OpenAI	United States	Multilingual (particularly strong in English contexts) and moderate performance	No	Requires a cloud API inference service	64.68
EmbeddingGemma	Google	United States	Open source, multilingual support, lightweight	Yes	Can be deployed in the cloud or on an on-premises GPU based on requirements	61.15
Mxbai-embed-large	Mixedbread AI	United States	Open source, excellent balance between performance and resources; strong long-context performance	Yes	Can be deployed in the cloud or on an on-premises GPU based on requirements	64.68
BGE-Large	BAAI	China	Open source, multilingual	Yes	Can be deployed in	64.23

Model	Developer	Country of Origin	Characteristics	Open Source	Deployment Method	Average MTEB Score
			support, lightweight		the cloud or on an on-premises GPU based on requirements	
Nomic-embed-text	Nomic AI	United States	Open source, lightweight	Yes	Can be deployed in the cloud or on an on-premises GPU based on requirements	62.39
Qwen3-Embedding 0.6B	Alibaba	China	Open source, lightweight	Yes	Can be deployed in the cloud or on an on-premises GPU based on requirements	61.82
Granite-embedding-278m-multilingual	IBM	United States	Open source, multilingual, lightweight	Yes	Can be deployed in the cloud or on an on-premises GPU based on requirements	56.1

To ensure accurate semantic representations and retrieval, MaiAgent selects and evaluates its Embedding models according to the MTEB (Massive Text Embedding Benchmark) standard. MTEB is a leading benchmark for comparing semantic vectorization models and covers a variety of task types, including: Retrieval (semantic retrieval) Classification Clustering Reranking STS (Semantic Textual Similarity) Summarization QA Pair Classification, and more

Advantages of MaiAgent's Embedding Technology

1. Flexible Model Selection:

Provides multiple Embedding model options to meet different requirements

2. Multiple Deployment Options:

Supports cloud and on-premises deployment to ensure data security

3. Performance Optimization:

Specially optimized for RAG scenarios to deliver the best retrieval results

4. Cost-Effectiveness:

Selects the appropriate model based on actual requirements to balance performance and cost

Reranker Models

What Is a Reranker?

A **Reranker** is a model or system used for **ranking tasks**, typically in information retrieval, recommendation systems, or natural language processing (NLP). Its primary purpose is to reorder a set of candidate results so that the most relevant or suitable options are returned.

How a Reranker Works

When a search engine or recommendation system returns a set of candidates—such as search results, recommended products, or articles—they have typically already been ordered according to preliminary ranking criteria. This initial ranking may be based on simple matching or basic relevance metrics. However, it may not fully account for every detail or deeper semantic relationships, which can produce results that do not completely meet the user's needs.

For example, suppose you search online for “the best pizza restaurants.” The search engine returns many results based on basic criteria such as restaurant names, ratings, and addresses. These results may already be ranked by keyword or simple matching, but they may not fully meet your needs.

At this point, a Reranker further helps you “**optimize the ranking.**” It examines the results using additional details, potentially considering your previous search habits, other customers' reviews, or the restaurants' latest status. It then reranks the results and selects the most relevant restaurants, helping you find the best option more quickly. In this example, the Reranker workflow is as follows:

Initial ranking: Use a simple ranking model (such as keyword matching, TF-IDF, or other features) to rank the candidates initially

Search for “the best pizza restaurants,” and the search engine returns a list of restaurants

Candidate refinement: Refine the ranking and rescore the candidates using additional features, such as contextual understanding, semantic relationships, and user behavior

Examine the restaurants more closely, considering whether they are in areas you visit frequently, whether other users have rated them highly, or whether they offer promotions

Final ranking: Return the most relevant or suitable results based on their scores

The Reranker places the restaurants that best meet your needs at the top, making it easier to find the most suitable option

Reranker Models Provided by MaiAgent

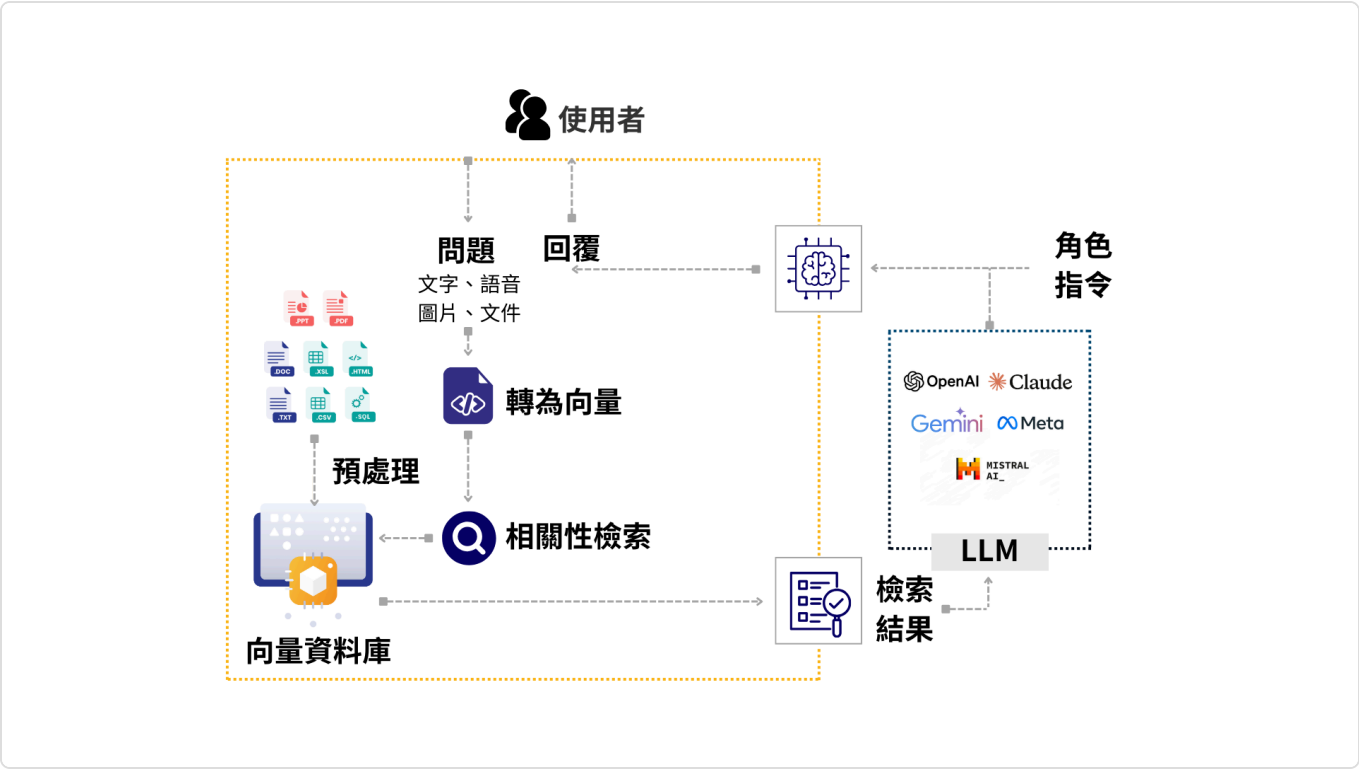
Features of Cohere Rerank v3.5

Cohere Rerank v3.5 is a **reranking** model provided by Cohere. It is designed specifically to optimize the ranking of search results or candidate answers. It precisely ranks candidates and uses deeper semantic and contextual understanding to select the most relevant results, thereby improving the final quality.

Parser Tools

What is a RAG Parser?

A RAG Parser is a key step in a Retrieval-Augmented Generation (RAG) system. It parses and breaks down source data before embedding, laying the foundation for vectorization and semantic retrieval. It has a decisive impact on data quality and retrieval results.



RAG workflow

1. Document Parsers

MaiAgent provides four document parsers for formats such as PDF, Word, Excel, and images:

Feature	MaiAgent Parser (default)	MaiAgent Parser (Online)	MaiAgent Parser (Offline)	Vision Parser
Cost	Low	High	Medium	High
Image content parsing	Cannot extract text from images	OCR extracts text from images only	OCR + AI understands image meaning	AI visual understanding; best results
Uses an LLM	No	Yes	Yes	Yes

Feature	MaiAgent Parser (default)	MaiAgent Parser (Online)	MaiAgent Parser (Offline)	Vision Parser
Text parsing quality	Standard	Good	Good (preserves structure well)	Good
Table parsing	Native extraction	AI extracts text from images	Preserves native structure (best); includes static image assets as well as visual understanding	AI visual recognition turns images into descriptive text
Parsing time	Fast	Medium	Slow	Slow
On-premises (offline)	Yes	No	Yes, with a deployed VLM	No
Supported formats	22	20	20 (OCR supports PDF only)	7

Auto Detect (Default) and Parsing Charges

When you upload PDF, Word (.docx), or PowerPoint (.pptx) files to a knowledge base, the parser dropdown beside each file defaults to **Auto Detect (Default)**. This does not mean the file will always use MaiAgent Parser. After upload, the system examines the file structure and selects a parser:

Files containing only text, with no significant image content → Use the default parser for that file type (usually MaiAgent Parser).

Scanned files (without a text layer), files containing image content (such as engineering drawings or scans), and PowerPoint files → The system chooses among parsers that support OCR or visual understanding (MaiAgent Parser (Online), MaiAgent Parser (Offline), or Vision Parser). If none is suitable, it uses the default parser.

The decision depends on file structure—whether there is a text layer and how many pages contain images or tables—rather than the number of images. A Word file with any embedded image counts as containing image content. Even a document with only a few images may therefore use one of the parsers above. Check the **Parser** column in the knowledge base document list for the parser actually used. Credits are charged according to **the parser that actually completed parsing**:

Parser	Billing method	Transaction history shows
MaiAgent Parser	Based on the number of parsed chunks	<code>Document parsing (maiaagent): filename</code>
MaiAgent Parser (Online)	Based on the number of billable pages; charged for each parsing run	<code>Document parsing (llama): filename</code>

Parser	Billing method	Transaction history shows
MaiAgent Parser (Offline)	Based on the number of parsed chunks	Document parsing (docling): filename
Vision Parser	Based on input/output tokens for the model configured for that parser	Vision parsing input: filename , Vision parsing output: filename

Credits are deducted only after file processing completes. Failed processing is not charged. For the same file, each parser type is charged at most once (all Vision Parsers count as one type). Switching to a parser that the file has not used before in the document list triggers parsing again and charges according to the new parser. Switching back to a previously used parser, rerunning the same parser, or switching between different Vision Parsers does not deduct credits again. Exception: MaiAgent Parser (Online) charges by actual billable pages for every parsing run, including switching back to it or rerunning it. No credits are deducted when the parsing service hits its cache and incurs no charge. Credits deducted for earlier parsing are never refunded, so the transaction history for one file may show several parsing items. Changing a parser in the document list (including [Bulk Change Parser](#)) immediately starts parsing again, without a confirmation dialog. To keep parsing costs predictable and prevent automatic switching to another parser, select MaiAgent Parser (or another specific parser) when uploading instead of [Auto Detect \(Default\)](#).

2. Speech-to-Text Parsers

MaiAgent provides four speech-to-text parsers that transcribe audio files for inclusion in a knowledge base:

Feature	Azure Speech	Whisper (Groq)	Whisper (OpenAI)	Whisper (Offline)
Cost	High	Low	Medium	Free
Transcription accuracy	High	High (large-v3)	High (whisper-1)	High (depends on local model)
Uses an LLM	No	No	No	No
Parsing speed	Real-time	Fastest	Medium	Depends on hardware
On-premises (offline)	No	No	No	Yes
Multilingual support	Yes	Yes (auto-detect)	Yes (auto-detect)	Yes (auto-detect)
Custom prompts	No	Yes	Yes	Yes

Feature	Azure Speech	Whisper (Groq)	Whisper (OpenAI)	Whisper (Offline)
VAD speech detection	—	Yes	Yes	Yes
Data privacy	Cloud (Azure)	Cloud (Groq)	Cloud (OpenAI)	Fully local

After an audio file is uploaded to a knowledge base, the system automatically transcribes it and lets you view and download the transcript:

Image Recognition Support

What is VLM (Vision Language Model)?

Vision Language Model (VLM) is an advanced artificial intelligence model that can understand image content and combine image information with textual information. Unlike traditional image processing technologies, VLM doesn't just "see" images, but can "understand" objects, scenes, relationships within images, and generate text, answer questions, or execute related commands based on image content.

The core capability of VLM lies in its multimodal processing ability - simultaneously processing and understanding information from different sources (visual and textual). This enables VLM to perform more complex and context-aware tasks.

Comparison between VLM and Traditional OCR

Traditional Optical Character Recognition (OCR) technology focuses on detecting and extracting text from images but has limitations in understanding the overall semantics and non-textual content of images.

Characteristic	Traditional OCR	VLM (Vision Language Model)
Main Function	Extract text from images	Understand image content and combine with textual information
Information Processing	Single modal (only processes text pixels)	Multimodal (processes both visual information and text semantics)
Understanding Level	Character-level recognition	Scene understanding, object recognition, relationship inference, context awareness
Main Tasks	Document scanning, text extraction	Image description generation, visual QA (VQA), image retrieval, object detection etc.
Context Awareness	Limited, mainly relies on language models in post-processing	Strong, can understand overall context and details of images
Non-text Content Processing	Usually ignored or unable to process	Can identify and understand objects, scenes, actions etc. in images

VLM's advantages include:

* **Deeper Understanding:** VLM not only reads text but understands semantic content of images, such as recognizing objects, analyzing scenes, understanding relationships between elements in images.

- * **Interactivity:** VLM can perform Visual Question Answering (VQA), answering user questions based on image content.
- * **Content Generation:** VLM can generate descriptive text for images (Image Captioning).
- * **Versatility:** Beyond text-related tasks, VLM can be applied to broader visual understanding tasks.

Real VLM Application Cases

VLM's powerful capabilities enable wide applications across multiple fields:

Visual Question Answering (VQA)

Application: Users upload an image and ask questions about its content, like "What color clothes is the person wearing?" or "Was this photo taken indoors or outdoors?"

Scenarios: Smart assistants, education, visual impairment assistance.

Image Captioning

Application: Automatically generate concise, accurate text descriptions for images.

Scenarios: Automated image annotation, content management systems, social media content generation, assisting visually impaired people understand images.

Content-Oriented Image Retrieval

Application: Allow users to search images using natural language descriptions, e.g., "Find images of people having meetings in offices"

Scenarios: Large image library management, e-commerce product search.

Multimodal Data Analysis

Application: Combine medical images with patient records to assist diagnosis; analyze product images and user reviews for market trend prediction.

Scenarios: Healthcare, retail, finance.

Human-Machine Interaction

Application: Enable robots or virtual assistants to understand their visual environment and interact more naturally with humans.

Scenarios: Smart robots, autonomous vehicles (understanding traffic signs and road conditions).

MaiAgent Image Recognition Advantages

MaiAgent RAG integrates VLM technology to provide technical personnel and developers with convenient and efficient image recognition and understanding solutions, with the following advantages:

Attachment Image VLM Recognition (Image Understanding and Q&A)

MaiAgent RAG can perform VLM recognition on user-uploaded attachment images, providing precise image content understanding.

VLM Recognition of Embedded Images in Attachment Documents

MaiAgent RAG is actively developing VLM recognition capabilities for embedded images in attachment documents (such as PDF, Word documents). This means the system can not only understand document text but also analyze image content, achieving true multimodal document understanding. This is an advanced feature many standard RAG systems lack.

Content Q&A for Document Images

Whether for attachment images or embedded document images, MaiAgent RAG supports content-based Q&A, allowing users to ask specific questions about image details and receive accurate answers.

Content Q&A for Knowledge Base Document Images

MaiAgent RAG supports image Q&A in knowledge base documents (requires Prompt under role instructions, displaying images in Markdown format).

Post-VLM Recognition Q&A for Knowledge Base Document Images

MaiAgent introduces an experimental feature that first performs VLM recognition on images in knowledge base documents, then combines recognition results for in-depth Q&A. This further bridges image information with knowledge base connections, providing more comprehensive knowledge services.

Through MaiAgent's VLM technology, you can more deeply mine the value of image information, achieving more intelligent human-machine interaction and automated processes.

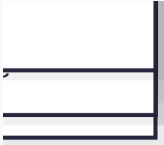
MaiAgent Work | Employee Productivity Integration

Add MaiGPT to Your System

This page is designed to be read from start to finish by engineers who need to add MaiGPT to their system. Whether you operate an enterprise portal, an internal workspace, or a SaaS platform managed for customers, the process is the same as long as users are **members of a MaiAgent organization** (such as employees or staff). Each step tells you exactly what to do, with links for deeper details when needed.

If your users are your customers, members, or anonymous visitors, rather than organization members, they cannot use the MaiGPT product itself. Use the [custom AI Agent + MaiGPT window mode](#) instead. For contact identity integration, see the [Software Vendor Integration Guide](#).

Overview



Mermaid 圖表

When complete, employees sign in to your system and MaiGPT appears directly on the page. They can generate images, use Deep Research, and access connectors. Their permissions, credits, and conversation history all follow their MaiAgent member identity. They do not need to sign in to MaiAgent separately or even know that MaiAgent is involved.

Prerequisites

Item	Where to get it
A MaiAgent organization with a plan that includes MaiGPT and identity synchronization	Contact sales if you are unsure

Item	Where to get it
An organization API Key owned by the organization owner or someone with the “Organization Overview” administration permission	See Quickstart for instructions
The MaiGPT Web Chat ID	In the response from <code>GET /api/v1/web-chats/</code> , find the entry whose name is <code>MaiGPT</code> and whose <code>accessType</code> is <code>web</code>
Roles created and permissions configured in advance	In the backend, go to “Organization Settings → Organization Overview → Edit Role Permissions.” Create a role such as “Employee” and enable permission to use MaiGPT (configuration instructions). All member capabilities come from their role. If you skip this step, none of the later steps will work
Decide where members will come from	Choose one of the three options in the next section

Step 1: Provision Members

Your users must first be members of the MaiAgent organization. Choose the easiest of these three approaches:

Are people in your organization already using MaiAgent? Connecting SCIM will not invalidate their existing accounts or create duplicate accounts. Existing members brought under management will subsequently be synchronized with your system. For details, see [What Happens to Existing Accounts and Members](#).

Your system manages users (recommended)

When users are created, updated, or deactivated in your system, call the MaiAgent SCIM endpoint as part of the same event to keep members synchronized in real time.

In the backend, go to “Organization Settings → Identity Synchronization,” select SCIM, and copy the Endpoint URL and Token from the “Connection Information” tab. The Token is shown only once; store it in a server environment variable.

Add the `MaiAgentMembers` client from the [real-time synchronization page](#) to your project and call it from the event:

```

members = MaiAgentMembers()    # Reads MAIAGENT_SCIM_URL MAIAGENT_SCIM_TOKEN from
                                environment variables

# Create user
user.maiagent_member_id = members.on_user_created(
    external_id=user.id, email=user.email, name=user.display_name, groups=
    ["Employee"],
)
# Rename or transfer department
members.on_user_updated(user.maiagent_member_id, user.id, user.email,
user.display_name, ["Employee"])
# Suspend access or offboard
members.on_user_deactivated(user.maiagent_member_id)

```

Before the initial launch, run `on_user_created` once for all existing users (it is idempotent; see [batch backfill](#)). After that, use events to keep them synchronized.

You only have a roster (HR export, Excel, or database)

Use the [roster synchronization tool](#). Prepare a four-column CSV or JSON file containing employee ID, Email, name, and role; configure the SCIM connection environment variables; run with `--dry-run` to review the plan before executing it; then schedule it to run once a day. Users present on the roster are created or updated, while users no longer on the roster are deactivated.

You use Okta, Entra ID, or JumpCloud

Enter the SCIM Endpoint URL and Token in your IdP's provisioning settings. The IdP will automatically push users and groups. For details, see [Push from an IdP](#).

Remember these three points. They are the most common sources of integration issues:

Role names must match roles created in the backend: The name supplied in `groups` when SCIM creates a member must match the role whose permissions were configured in the backend. If the name does not match, a new role **without permissions** is created automatically.

Email is the account key: `userName` is the Email later used to issue a token. Email cannot be changed through SCIM after the account is created.

Accounts created through SCIM do not have passwords: Employees following this path will not sign in to MaiAgent, which is intentional. If you also want to provide access to the MaiGPT web version or app, configure [SSO](#) separately.

Step 2: Issue a Member Token in the Backend

Create an endpoint in your backend that can be called only by **signed-in users** (for example, `GET /maigpt-token`). Internally, use the organization API Key to request a short-lived token for that member from MaiAgent:

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
-H "Authorization: Api-Key <your organization API Key>"
-H "Content-Type: application/json"
-d '{"email": "alice@acme.com"}'
```

A successful request returns `201`:

```
{
  "accessToken": "<short-lived member JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": { "id": "c1a4 ... ", "userId": "5e77 ... ", "email": "alice@acme.com",
  "name": "Alice" }
}
```

Return the entire response to the frontend unchanged. Rules and status codes:

Fact	Description
Valid for 15 minutes; no refresh token is issued	The frontend SDK handles renewal automatically (Step 3). Your backend only needs to “issue one whenever called” — always issue a new token; do not cache it
The token can be used only with MaiGPT conversation endpoints	Using it with any <code>/api/v1/</code> API always returns <code>401</code> ; even if leaked, it cannot access organization data
<code>401</code> <code>403</code>	No Api-Key was used, or the Key owner is not an owner or organization administrator
<code>404</code>	The Email does not belong to an active member of this organization—the first step was not completed, or the member has been deactivated
Every issuance is recorded in the audit log	It can be reviewed in the organization's API request logs; the token in the response is not logged

For parameters and more details, see [Issue a Member Token](#).

Step 3: Embed in the Frontend

Add the embed code to the page where MaiGPT should appear, and point `auth.getToken` to the endpoint created in Step 2. The SDK handles everything else: it retrieves a token during initialization, automatically retrieves another before expiry, and continues using the same conversation.

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',           // Optional: brand name in the upper-left
corner
    locale: 'zh-TW',                  // Optional: interface locale
    auth: {
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' })
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

sourceId must be set to member.userId from the response—not member.id or the user ID from your own system. MaiGPT uses it to associate conversations with the member. If it is incorrect, conversations become anonymous and the member's history will not be visible.

Set the container height yourself. In an SPA, wait until the container is mounted before loading the SDK. `baseUrl` and the SDK URL point to the SaaS service; for a private deployment, replace them with your domain. For layout and other parameters, see [Embed in MaiGPT Mode](#).

Step 4: Sign-Out and Offboarding

Event	What you need to do	Result
A user signs out of your system	Call <code>MaiAgent.auth.signOut()</code> in the frontend	The iframe clears the identity and stops renewal; an issued token may remain valid for up to 15 minutes

Event	What you need to do	Result
A user leaves or access is suspended	Deactivate the user using the Step 1 method (<code>on_user_deactivated</code> , removal from the roster, or deactivation in the IdP)	Subsequent issuance returns <code>404</code> , and the embedded page naturally stops working; conversation and credit records are retained
A user returns	Reactivate the user using the Step 1 method	The role and history are restored

Step 5: Acceptance Testing

Verify each item in order. Every item has a clear pass condition:

The member exists and has a role: In “Member Management” in the backend, confirm that the person appears and has the role whose permissions you configured.

Issuance succeeds: The curl command from Step 2 returns `201` ; using an Email that does not exist returns `404` .

Conversation works: Open MaiGPT on the embedded page and send a message; verify that it receives a response. Sign in to the MaiGPT web version as that member (if SSO is configured), or check the backend conversation records to confirm the conversation appears.

Capabilities are correct: Ask it to generate an image. If the role has access to connectors and Deep Research, those options appear.

Renewal is seamless: Leave the page open for more than 15 minutes, then send another message. The same conversation continues without reloading the page.

Sign-out takes effect: After `signOut()` , attempts to send another message are rejected. After deactivating the member, Step 2 returns `404` .

After Launch

Rotate the SCIM Token and API Key: The backend displays a reminder after a SCIM Token is more than 180 days old. When a new one is generated, the old one is immediately invalidated, so remember to update the server environment variable at the same time.

Plan capacity: The number of members affects built-in Agent seats and per-member Credits. See [Manage Member Credit Quotas](#).

Auditing: Every token issuance appears in the organization's API request logs. Every SCIM synchronization appears under “Identity Synchronization → Synchronization Records” in the backend.

Troubleshooting

Symptom	Cause	Resolution
Issuance returns <code>404</code>	The Email does not belong to an active member of this organization	Return to Step 1 and confirm that the member exists, is active, and has the same Email (case-insensitive)
Issuance returns <code>403</code>	The API Key owner is neither an owner nor an organization administrator	Use a Key issued by an administrator
The chat window says “Failed to create conversation (400): You do not have permission to use MaiGPT”	The member's role does not have permission to use MaiGPT, or <code>auth</code> was not provided	Enable MaiGPT in the backend role permissions; confirm that <code>getToken</code> returns <code>accessToken</code> and <code>organizationId</code>
The conversation is anonymous and history is unavailable	<code>sourceId</code> is incorrect	Use <code>member.userId</code>
Sending a message fails after 15 minutes	<code>getToken</code> was not called or failed (check the browser console for <code>[MaiAgent]</code> errors)	Confirm that the endpoint is available to signed-in users and that its response format is correct
The member was created but cannot use anything	The member has no role, or the role has no permissions	Configure role permissions in the backend; assign the same role name through SCIM
An Email change did not take effect	SCIM does not allow Email changes	Change it in the backend, or deactivate the old account and create one with the new Email

For more SCIM error codes, see the [roster synchronization tool error reference](#).

Further Reading

[Member Provisioning: SCIM](#)

[Invitation](#)

[SSO](#) — Complete SCIM API reference and comparison of the three approaches

[Real-Time Synchronization: Call from User Events](#)

[Roster Synchronization Tool](#)

[Integrate MaiGPT: Choose the Right Assistant and Enable Capabilities](#) — MaiGPT product vs. custom assistants and complete parameters for embedding member identity

[Web Chat Embedding and SDK](#)

[Embed in MaiGPT Mode](#)

Integrate MaiGPT: Choose the Right Assistant and Add the Capabilities You Need

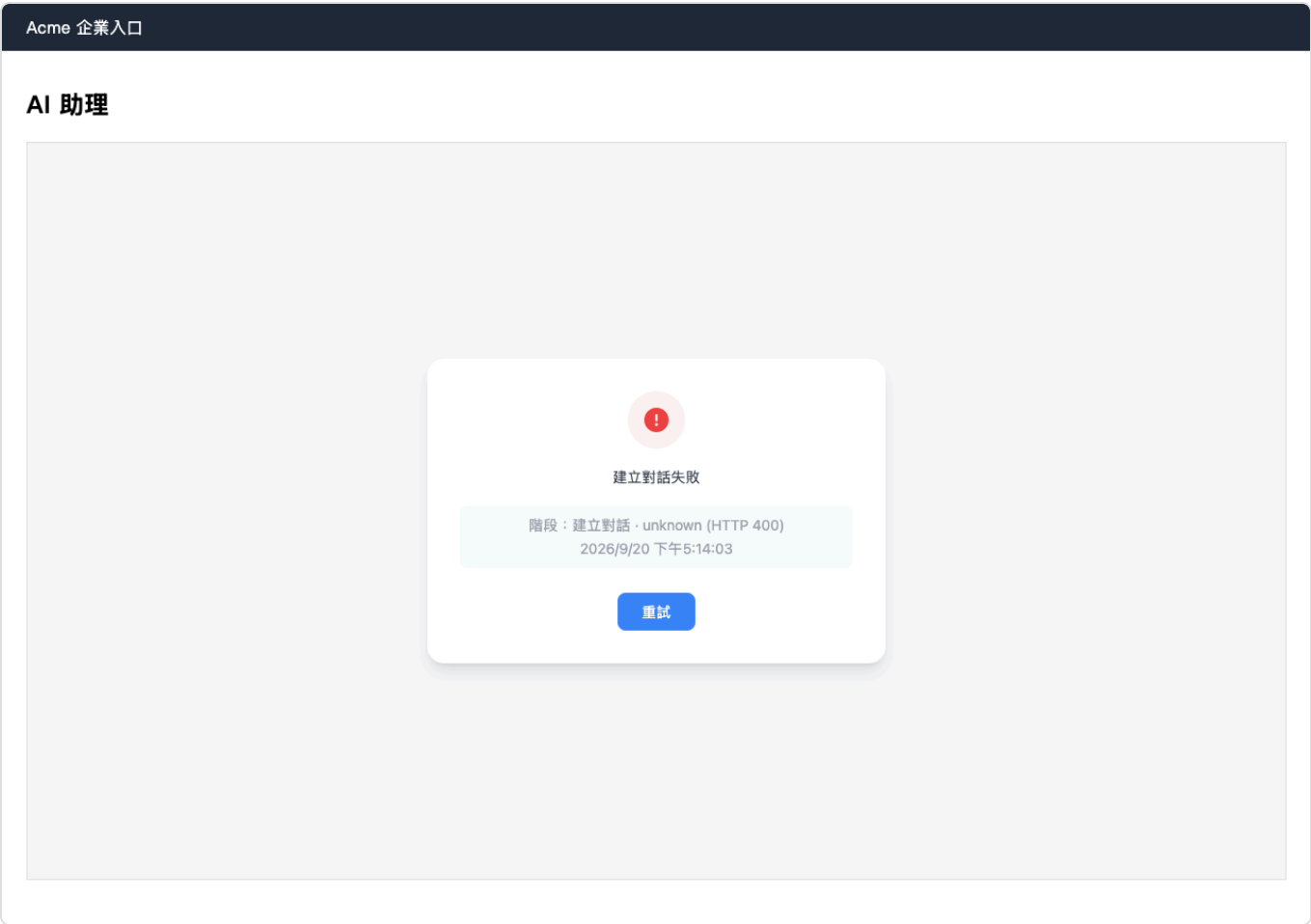
On the MaiAgent platform, “MaiGPT” refers to two different things. Distinguishing them before integration will save most of the back-and-forth:

	MaiGPT product (organization's built-in agent)	MaiGPT window mode (a Web Chat SDK layout)
What it is	An enterprise AI workspace automatically provided to every organization and centrally maintained by MaiAgent. It includes AI image generation, Deep Research, connectors, and other capabilities. See What is MaiGPT?	A full ChatGPT-like interface enabled with <code>enabledWindowModes: ['maigpt']</code> , including a conversation history sidebar, search, and settings. It is purely a frontend presentation mode.
Who can use it	Organization members after signing in (web, iOS App, LINE, and Microsoft Teams). Access is controlled by role permissions .	Any Web Chat can use it, including anonymous visitors and identified contacts .
Can it be embedded in your system?	Only for organization members: creating a conversation requires a member identity with the <code>maigpt_access</code> permission. Embedding it directly with its Web Chat ID returns <code>400 You do not have permission to use MaiGPT.</code> To embed it for members, use member identity embedding , with your backend issuing member tokens.	Yes: this is covered in Sections 2 and 3 of this page.
Can it be customized?	No: instructions, models, tools, and skills are configured centrally by the platform. Organizations can only restrict access through role permissions.	Yes: the interface runs an AI Agent you create, so you control its instructions, models, tools, and knowledge bases.

Conclusion: There are two use cases. For **your customers or other non-organization members**, **create your own AI Agent, add the required tools, and embed it using MaiGPT window mode** (Sections 2 and 3). For **organization members** who need the capabilities of the MaiGPT product itself—image generation, Deep Research, connectors, and role permissions—use [member identity embedding](#) (Section 5), with your backend issuing short-lived member tokens.

1. Common Symptoms

What you see	Cause	Solution
The interface looks like MaiGPT, but responses behave like regular Chat. When asked to generate an image, the AI says, “I can't generate images.”	A regular AI Agent is embedded, and either (1) its reply mode is not Agent mode , or (2) it does not have an image generation tool or other required tools.	Follow Section 2 to switch to Agent mode and add tools.
The chat window does not open and shows “Failed to create conversation (HTTP 400),” while the API returns <code>You do not have permission to use MaiGPT.</code>	The embedded Web Chat ID belongs to the organization's built-in MaiGPT product, but the visitor does not have a member identity.	For customers → use the Web Chat ID of a custom agent (Section 3); for members → provide a member token as described in member identity embedding .
The AI Agent list API in the admin console does not return MaiGPT.	<code>GET /api/v1/chatbots/</code> only lists agents created by the organization; the MaiGPT product is not included.	This is expected. Integrations should always target a custom agent.



Embedding the organization's built-in MaiGPT Web Chat: the visitor is not a signed-in member, so conversation creation is rejected (HTTP 400)

2. Create an AI Agent That Can Generate Images and Use Tools

The steps below use AI image generation as an example. Add other capabilities—web search, web fetch, MCP tools, or your own API tools—in the same way.

Create an AI Agent

In the admin console, go to “AI Features → AI Assistants,” create a new AI Agent, and configure its role instructions and model. See [Create an AI Assistant](#) in the user guide.

Switch the Reply Mode to “Agent Mode”

Open the agent's “Reply Mode Settings” and change the reply mode to **Agent mode**.

Only Agent mode invokes tools. The default “RAG Q&A” mode responds quickly but does not use any tools—the most common reason an agent still does not generate images after an image tool has been added.

Add Tools

Open the agent's “Tools” page, click “Select Tools,” and select the required built-in tools, such as:

Image generation: `Global_Image_Tool_OpenAI_GPT_Image_2`, `Global_Image_Tool_Gemini_3`, and others (select one; see [Built-in AI Image Generation Tools](#) for model differences)

Web search: `Global_Tavily_Search_Tool`

Web fetch: `Global_Web_Fetch_Tool`

An [API tool](#) for your own system or an MCP tool

For complete instructions, see [Configure Tools for an AI Assistant](#) in the user guide.

Create a Web Chat Conversation Platform

Under “Customer Service Conversations → Conversation Platforms,” add a **Web Chat** for this agent and obtain its `webChatId`. See [Integrate a Conversation Platform: Website](#).

Automate with the API (for ISVs Provisioning on Behalf of Customers)

Each of the four steps above has a corresponding API, allowing software vendors to complete provisioning programmatically when onboarding each customer. See [Quick Start](#) for authentication and instructions on obtaining an API Key.

```

# 1. List built-in tools and obtain the ID of an image generation tool
GET /api/v1/tools/

# 2. Create an AI Agent: use Agent reply mode and attach tools
POST /api/v1/chatbots/
{
  "name": "Acme GPT",
  "instructions": "You are Acme's AI assistant. Please respond in English.",
  "largeLanguageModel": "<LLM id>",
  "embeddingModel": "<embedding model id>",
  "rerankerModel": "<reranker model id>",
  "replyMode": "agent",
  "tools": ["<ID of Global_Image_Tool_OpenAI_GPT_Image_2>"]
}

# 3. Create a Web Chat conversation platform for the agent (channel.id in the
response is the webChatId)
POST /api/v1/inboxes/
{
  "name": "Acme GPT",
  "chatbot": "<chatbot id>",
  "channelType": "web",
  "channel": {
    "name": "Acme GPT",
    "theme": { "primaryColor": "#1890ff", " ... ": " ... " },
    "darkTheme": { "primaryColor": "#404040", " ... ": " ... " }
  }
}

```

To change the mode or add tools to an existing agent, use `PATCH /api/v1/chatbots/{id}`

. Pass the complete array of tool IDs in `tools` (the entire array is replaced).

`theme` and `darkTheme` are required. To obtain the field list, first use `GET /api/v1/web-chats/{existing webChatId}`

and copy an existing configuration.

Always create a Web Chat through `POST /api/v1/inboxes/` together with its inbox. Calling `POST /api/v1/web-chats/` directly creates an orphaned Web Chat without an inbox, which returns 404 when embedded.

3. Embed Using MaiGPT Window Mode

Take the `webChatId` from Section 2 and place the embed code from [Embed with MaiGPT Mode](#) on your page:

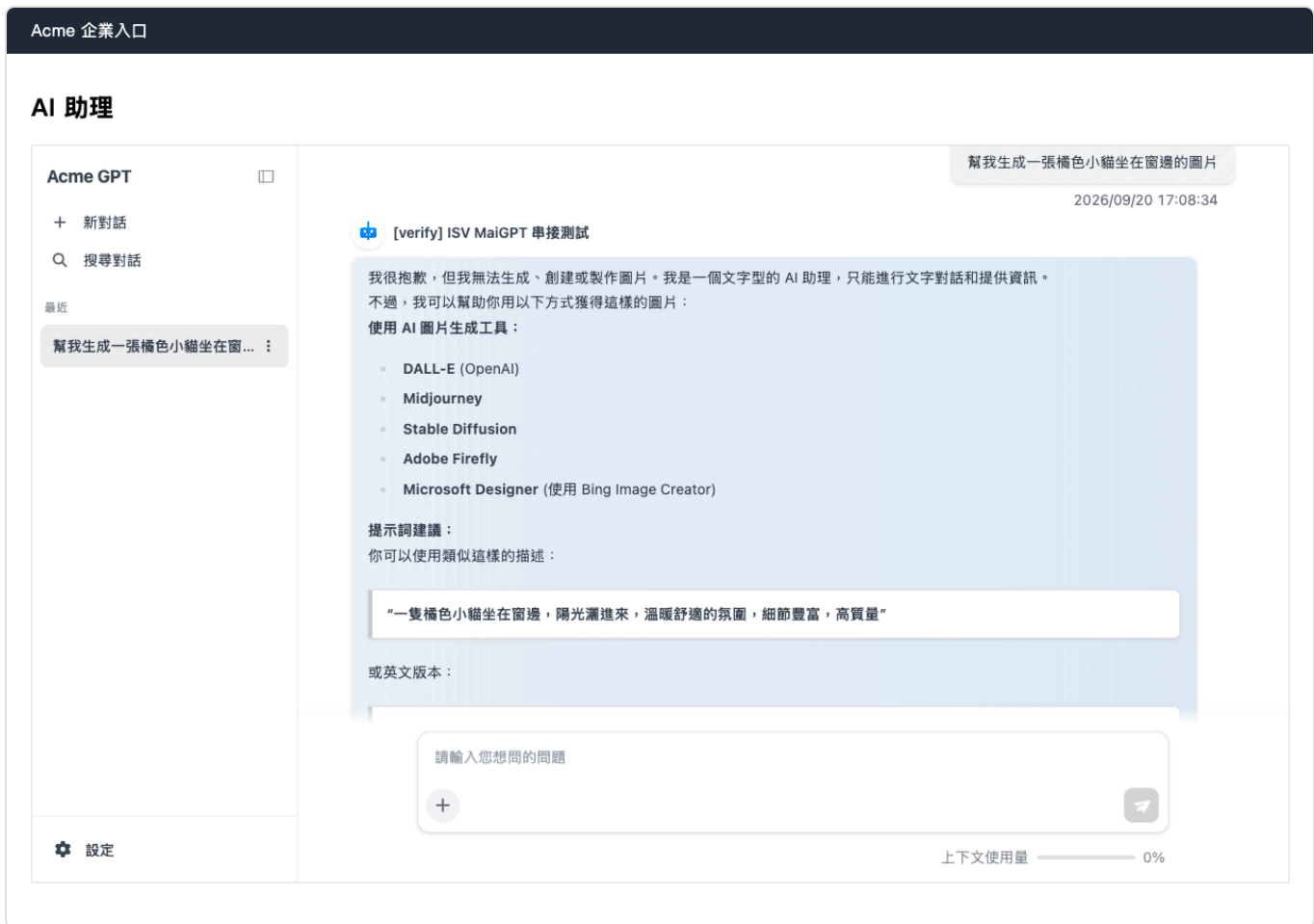
```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiaagentChatbotConfig = {
    webChatId: '<webChatId created in Section 2>',
    baseUrl: 'https://chat.maiaagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    maigptTitle: 'Acme GPT',
  }
</script>
<script src="https://chat.maiaagent.ai/js/embed.min.js" defer></script>
```

To let the agent identify signed-in users and call your APIs with their permissions, add `contactId` or `auth` as described in the [Software Vendor Integration Guide](#).

Verify: The Same Embed Code Before and After Adding Capabilities

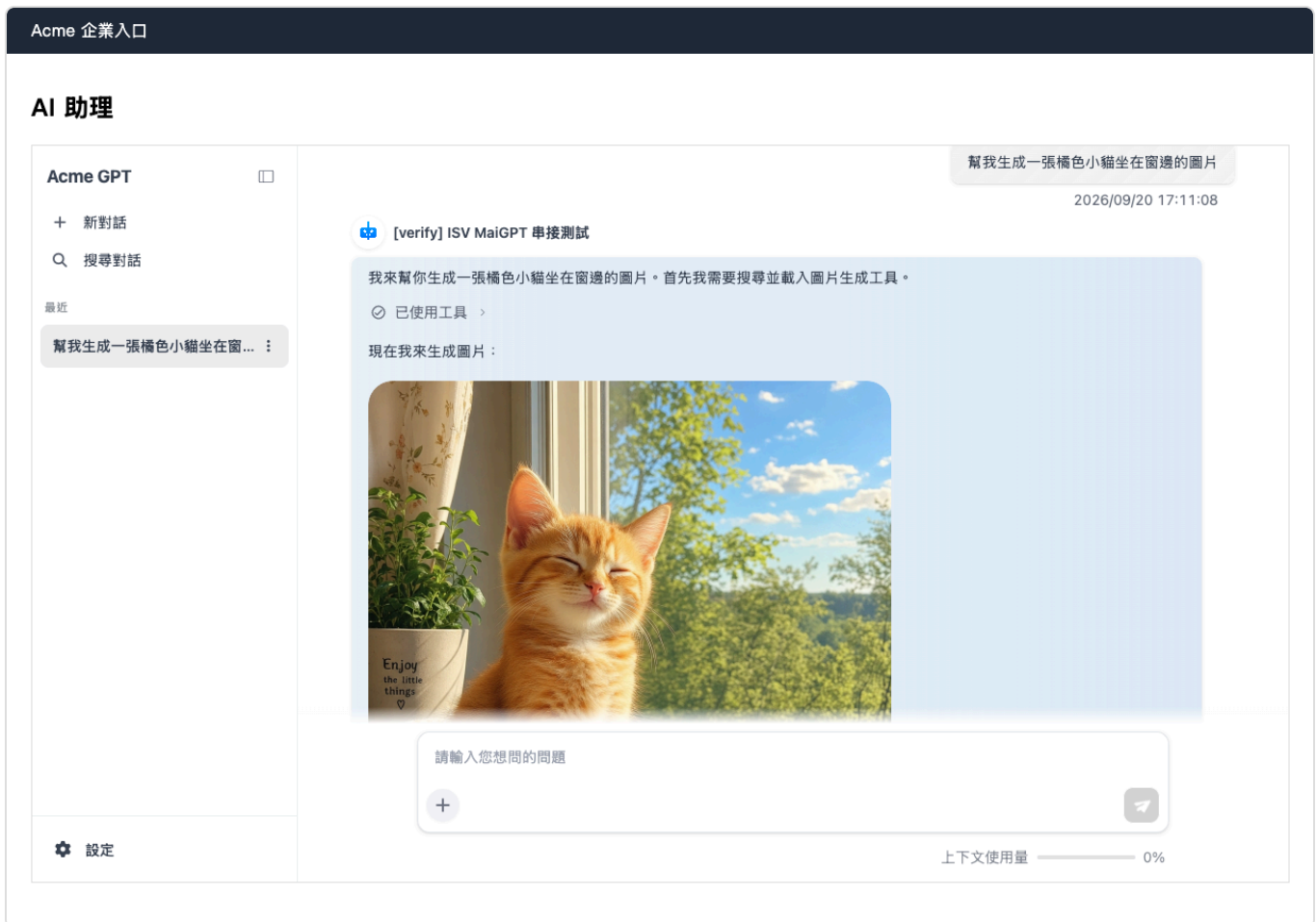
The two images below show the same AI Agent with the same embed code. The only difference is the “reply mode + tools” configuration:

Before: RAG Q&A mode without tools



The interface uses MaiGPT window mode, but the agent says, “I can't generate, create, or produce images.”

After: Agent mode with an image generation tool



With the same embed code, the agent invokes the image generation tool and displays the image directly in the conversation.

4. When to Use the MaiGPT Product Itself

If you need “a ChatGPT-level internal assistant for our employees,” rather than “an assistant embedded in our product for our customers,” using the MaiGPT product directly is the simplest option:

Employees can use it after signing in as organization members. Entry points include the [web app](#), [iOS App](#), [LINE](#), and [Microsoft Teams](#).

Image generation, Deep Research, connectors, and skills are built in. Administrators only need to use [role permissions](#) to determine who can use each capability.

To add organization-specific knowledge, make the relevant knowledge bases visible to members. You do not need to create a separate agent.

To embed MaiGPT in your own system when users are organization members, follow the member identity embedding flow in Section 5. If any of the following apply— “for non-members (customers or subscribers),” “needs your own API tools,” or “requires custom instructions or models” —use the custom-agent approach in Sections 2 and 3.

5. Member Identity Embedding: Embed the MaiGPT Product in Your System for Organization Members

For an end-to-end walkthrough, see [Add MaiGPT to Your System](#). This section provides a complete reference for each parameter.

Use this approach when users of your system **are also members of the MaiAgent organization**—for example, in an enterprise portal, an employee workspace, or an ISV-managed internal enterprise system—and they need the MaiGPT product itself rather than a separate agent. **Your backend** uses an organization API Key to issue a short-lived token for the signed-in member. The frontend then embeds the MaiGPT Web Chat using the SDK's existing member identity injection settings.

```
sequenceDiagram
    participant U as Member browser
    participant P as Your backend
    participant M as MaiAgent API
    participant W as MaiGPT Web Chat

    U->>P: Signed in; opens a page containing MaiGPT
    U->>P: SDK calls auth.getToken → your backend
    P->>M: POST /api/v1/members/embed-tokens/ (Api-Key, member email specified)
    M-->>P: accessToken (15 minutes), organizationId, member.userId
    P-->>U: Return data to getToken
    U->>W: SDK creates conversations and sends messages as the member
    Note over U,W: Before the token expires, the iframe notifies the SDK, which calls getToken again to renew it automatically
```

Mermaid 圖表

Prerequisites

An **organization API Key** whose owner is the organization owner or has the “Organization Overview” management permission. Only a key issued by someone who can manage members can issue tokens on their behalf. See [Quick Start](#) for instructions on obtaining an API Key.

The user to embed is already a member of the organization and has MaiGPT access through their role ([MaiGPT Role Permissions](#)). Token issuance returns 404 if the member does not exist; it does not provision the member automatically. For member provisioning options—SCIM provisioning API, admin invitation, and SSO—see [Member Provisioning](#). The standard approach for an ISV-managed deployment is to provision members through SCIM and then issue tokens as described in this section.

The MaiGPT Web Chat ID: in the response from `GET /api/v1/web-chats/`, use the entry named `MaiGPT` whose `accessType` is `web`.

Step 1: Issue a Member Token from the Backend

Create a backend endpoint that only signed-in users can call—for example, `GET /maigpt-token`. It calls MaiAgent internally, and the frontend SDK calls it whenever needed:

```
curl -X POST https://api.maiagent.ai/api/v1/members/embed-tokens/
-H "Authorization: Api-Key <your organization API Key>"
-H "Content-Type: application/json"
-d '{"email": "alice@acme.com"}'
```

Provide either `email` or `memberId`, but not both (`memberId` is the MaiAgent member ID). A successful request returns `201` (`expiresAt` is an epoch timestamp in milliseconds):

```
{
  "accessToken": "<short-lived member JWT>",
  "organizationId": "8d2f ... - ... ",
  "expiresAt": 1789906401000,
  "member": {
    "id": "c1a4 ... - ... ",
    "userId": "5e77 ... - ... ",
    "email": "alice@acme.com",
    "name": "Alice"
  }
}
```

Status code	Meaning
400	Both <code>email</code> and <code>memberId</code> were provided, or neither was provided.
401	The request was not authenticated with an Api-Key (member sign-in tokens and anonymous requests are rejected).
403	The API Key owner is neither the organization owner nor a user with organization management permission.
404	The specified member does not exist, is disabled, or does not belong to the organization associated with the key.

Token rules: The lifetime is fixed at 15 minutes. No refresh token is issued, and the lifetime cannot be extended. You can issue tokens for the same member repeatedly; earlier tokens remain valid until they expire. This token can only be used for Web Chat conversations (the three calls for creating conversations, sending messages, and continuing after handoff). Using it for any `/api/v1/` endpoint, any other conversation management action, or WebSocket access always returns 401. Every issuance is recorded in the organization's API request logs for auditing (the token in the response is not logged).

Step 2: Provide the SDK with a Token Retrieval Function

The embed configuration is the same as in [Section 3](#), with an additional `auth.getToken`. Replace `webChatId` with the MaiGPT Web Chat ID:

```
<div id="maigpt-container" style="height: 720px;"></div>
<script>
  window.maiagentChatbotConfig = {
    webChatId: '<MaiGPT Web Chat ID>',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    enabledWindowModes: ['maigpt'],
    targetElement: '#maigpt-container',
    auth: {
      // Called once during SDK initialization and again whenever the token is
      // about to expire; renewal is automatic, so you do not need a timer
      getToken: async () => {
        const res = await fetch('/maigpt-token', { credentials: 'include' }) //
        Your backend; see Step 1
        const { accessToken, organizationId, member } = await res.json()
        return { accessToken, organizationId, sourceId: member.userId }
      },
    },
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

Fields returned by `getToken`:

Field	Required	Description
<code>accessToken</code>	✓	The <code>accessToken</code> returned in Step 1.
<code>organizationId</code>	✓	The <code>organizationId</code> returned in Step 1.
<code>sourceId</code>	Recommended	The <code>member.userId</code> returned in Step 1. You can also set it at <code>auth.sourceId</code> ; if both locations provide it, <code>auth.sourceId</code> takes precedence.

`auth` must be defined in `maiagentChatbotConfig` so the SDK knows to wait for member identity injection before creating the first conversation. If you call `MaiAgent.auth.setup()` only after the SDK has loaded, the first conversation may be created anonymously before identity injection and be rejected.

sourceId must be the member.userId from the response—not member.id or the user ID from your own system. MaiGPT uses it to associate conversations with members. If it is incorrect, the conversation becomes anonymous and the member cannot see their MaiGPT history.

How Expiration and Renewal Work

Tokens expire after 15 minutes. The iframe notifies the SDK 60 seconds before expiration. The SDK calls `getToken` again to obtain and inject a new token, and **continues the same conversation without reloading the page or iframe**, transparently to the user.

Every `getToken` call must request a new token from MaiAgent; do not return a cached value. The SDK schedules the next renewal based on the token's own remaining lifetime. Returning the same token as before is treated as a failure and triggers the retry flow described below.

If `getToken` fails—for example, because the user has signed out or the backend returns 401—the SDK retries after 30, 60, and 120 seconds. If all retries fail, it stops and logs an error to the console. After the token expires, the next message fails to send and displays the existing error notification. The conversation and previously sent messages remain visible.

When the user signs out of your system, call `MaiAgent.auth.signOut()`. The SDK clears the identity and stops renewal. Any issued token may remain valid for up to 15 minutes. When another user signs in, call `MaiAgent.auth.setup()` again; any renewal in progress for the previous user is invalidated.

If the Web Chat has Source ID signature verification enabled, the signature is single-use. The SDK does not reuse the initial `ts`

`sig` during renewal, so have `getToken` return a new signature along with the token (`{ accessToken, ts, sig }`).

If you provide a static `auth.accessToken` instead of using `getToken`, the SDK does not renew it. You must call `MaiAgent.auth.setup()` again after 15 minutes.

Choosing Between These Approaches

	Embed MaiGPT with a member identity (this section)	Custom agent + MaiGPT window mode (Sections 2 and 3)
Users	Must be organization members	Anyone (customers, subscribers, or anonymous visitors)
Capabilities	Complete MaiGPT feature set: image generation, Deep Research, connectors, and skills, subject to role permissions	The tools and knowledge bases you select
Custom instructions/models	No (centrally managed by the platform)	Yes

	Embed MaiGPT with a member identity (this section)	Custom agent + MaiGPT window mode (Sections 2 and 3)
Backend work	One endpoint that proxies the token issuance API	None (optional contact identity synchronization)
Billing attribution	The member's quota	The agent/Web Chat

6. Frequently Asked Questions

- I see a Web Chat named MaiGPT under “Conversation Platforms” in the admin console. Can I use its embed code?
- Can a custom agent use the image generation, Deep Research, and connector capabilities available in the MaiGPT product?
- The agent is already in Agent mode and has an image generation tool, but it still does not generate images. What should I check?
- How does MaiGPT window mode differ from floating/sidebar mode?

Member Memory Management API

Signed-in members can view and manage their own memories through `/api/me/memories/`. Include `X-Organization-Id` in each request to specify the organization being accessed. The service retrieves data according to the signed-in user's active membership in that organization. For sign-in and common request conventions, see [Quickstart](#).

Available operations

Operation	Endpoint	Behavior
List	<code>GET /api/me/memories/</code>	List the current member's <code>active</code> and <code>expired</code> memories. Filter with the <code>status</code> and <code>kind</code> query parameters.
Create	<code>POST /api/me/memories/</code>	Create a memory for the current member. If this member already has an active memory with the same <code>topic</code> , update it instead. Creation returns HTTP 201; an update returns HTTP 200.
Edit or reactivate	<code>PATCH /api/me/memories/{id}</code>	Edit a visible memory, or set its <code>status</code> to <code>active</code> or <code>expired</code> .
Delete	<code>DELETE /api/me/memories/{id}</code>	Delete a specified memory belonging to the current member.

When creating a memory, `topic`, `summary`, and `details` are required. `topic` can have at most 100 characters and can contain only lowercase English letters, digits, and underscores between characters. `summary` can have at most 500 characters. `details` can contain at most 20 items, each with at most 500 characters. `kind` can be `fact`, `preference`, or `instruction`; it defaults to `fact` if omitted.

List and individual operations show only active and expired memories. Memories awaiting confirmation or rejected memories do not appear in this member interface.

Status handling in integrations

If a member marks a memory as `expired`, an integration can still see it in the list and use `PATCH` to change its status back to `active`. Creating a memory without an active membership returns HTTP 404. Integrations should display memories under the organization specified in the request to avoid mixing data from different organizations in one view.

Software Vendor Integration Guide: Embed AI Agents for Your Customers

This page is for software vendors that want to embed MaiAgent Web Chat in their products and integrate it with their own account and permission systems. After users sign in to your system, the AI Agent knows who they are, can retrieve their conversation history, and calls your system's APIs with **their permissions**.

The integration journey consists of five stages. This page explains what to do at each stage and in what order, with links to the corresponding technical documentation for details.

Integration Overview

```
flowchart LR
    subgraph ONCE ["One-time setup"]
        A["Embed Web Chat"] --> B["Batch-provision existing users"]
    end
    subgraph DAILY ["Daily operations (account lifecycle)"]
        C["Create a contact when adding a user"] --> D["Update the token at login"]
        D --> E["Synchronize again when the token is refreshed"]
        E --> F["Revoke credentials at logout"]
    end
    ONCE --> DAILY
```

Mermaid 圖表

The users on this page are your customers (represented as MaiAgent contacts). If you need to integrate employees of your company or your customers and make them MaiAgent organization members who can use MaiGPT, role permissions, and Credits, see [Member Provisioning: SCIM, Invitations, and SSO](#).

There is only one core concept: **Contact**. Each user in your system corresponds to one contact in MaiAgent. Conversation history, personalized attributes, knowledge base query scope, and tool credentials are associated with the contact. Every permission-integration action ultimately maintains the mapping between your user and the contact, along with the contact's credentials. For an introduction to this concept, see [Introduction to Contacts and Integration](#).

Frontend and Backend Responsibilities

Throughout the journey, your frontend is only responsible for loading the chat window. Your backend makes every request that includes a token or API key:

```
graph LR
    subgraph FE ["Your frontend"]
        A["Embed code loads the SDK  
config includes contactId"]
    end
    subgraph BE ["Your backend"]
        B["Call signOut at logout"]
        C["Login/refresh: POST setup-contact-credentials"]
        D["Batch provisioning: POST bulk-import"]
        E["Data synchronization: PATCH contacts"]
        F["Logout: DELETE tool credentials"]
    end
    subgraph MA ["MaiAgent"]
        G["Chat window  
chat.maiagent.ai"]
        H["API  
api.maiagent.ai"]
    end
    A --> G
    B --> G
    C --> H
    D --> H
    E --> H
    F --> H
```

Mermaid 圖表

The `setup-contact-credentials` endpoint does not require an API key because it is public. `bulk-import`, `PATCH`, and `DELETE` all require `Authorization: Api-Key` authentication and must only be called from the backend.

Step 1: Embed Web Chat

First decide which AI Agent to embed. The embedded capabilities—such as image generation, web access, and tool calling—depend on the assistant settings behind Web Chat, not on the embedding method. The built-in organization product "MaiGPT" is only available to signed-in members, and its Web Chat cannot be embedded in an external system. Create an AI Agent for your product, set it to Agent mode, and attach the required tools. See [Integrate MaiGPT: Choose the Right Assistant and Add the Required Capabilities](#).

Choose an embedding method based on your product format. Both methods use the same SDK and contact mechanism:

Format	Best for	Documentation
Chat bubble (floating/sidebar)	Supporting features such as customer service and consultation	Web Chat Embedding and SDK
MaiGPT mode (full ChatGPT-like interface)	Main page areas and site-wide AI entry points	Embed in MaiGPT Mode

For Web Chat with identity integration, we recommend configuring the allowed embedding domain list at this stage. The MaiAgent team currently configures it for you, so provide your domain list to your integration contact. Restricting the list limits which origins can call the identity synchronization API. See [Allowed Embedding Domains](#).

Step 2: Initial Integration—Batch-Provision Existing Users

When integrating an existing system for the first time, provision contacts for all existing users in one batch. There are two options:

Method	Best for	Documentation
Batch import with Excel (<code>POST /contacts/bulk-import/</code>)	Large numbers of users (up to 10,000 rows per request), including knowledge query scopes and API tool credentials	Batch Import Contacts
Call the identity synchronization API for each user	Progressive provisioning through the login flow or small user counts	Batch-Provision Existing Users

It is fine if users' access tokens are not available during provisioning. Create the mappings first; the login flow in Step 4 will add token credentials the next time each user signs in.

Step 3: Create a Contact When Adding a User

After provisioning is complete, add an API call to your system's account creation flow so that every new user receives a corresponding contact immediately:

From the backend, call the `identity_synchronization` API (`setup-contact-credentials`), using your system's user identifier as the `sourceId`.

Store the returned `contactId` in your member database. For a recommended field design, see [Where to Integrate with the Login Flow](#).

`sourceId` is an idempotency key. Repeated calls with the same `sourceId` update the existing record instead of creating duplicates, so this step can safely coexist with Steps 2 and 4.

Step 4: Login and Token Refresh

After every successful login and creation of an access token, have the backend call the same identity synchronization API again with the **new token** (`mcpCredentials`). This enables the AI Agent to call your system's APIs with that user's identity and permissions. Use the same approach when refreshing the token: call the API again. The same contact-and-tool pair is updated instead of duplicated. For a complete sequence diagram and important considerations, including asynchronous calls and preventing synchronization failures from blocking login, see [Contact Identity Synchronization and Token Updates](#).

On the frontend, include the `contactId` when initializing Web Chat on the page. See [Identify Users](#).

If the AI does not need to call your system's APIs and only needs to recognize the user, you can skip the backend entirely and include auth in the frontend config. See [Choosing Between Frontend auth and Backend Integration](#) for the tradeoffs.

Step 5: Logout and Credential Revocation

Perform both of the following at logout. They have different scopes:

Frontend: Call the SDK's `signOut()` method to return the chat window to an anonymous state.

Backend: Delete the contact's tool credentials so the AI can no longer call tools as that user.

For detailed APIs and a comparison of these operations, see [Logout and Credential Revocation](#).

Integration Checklist

Stage	Action	Documentation
Setup	Create an AI Agent: enable Agent mode, attach image-generation and other tools, and create Web Chat	Integrate MaiGPT: Choose the Right Assistant and Add the Required Capabilities
Setup	Embed Web Chat (standard or MaiGPT mode)	Web Chat Embedding and SDK MaiGPT Mode
Setup	Give MaiAgent your domain list for the allowed embedding domain configuration	Allowed Embedding Domains
Setup	Add a <code>maiaagent_contact_id</code> field to the member database	Where to Integrate with the Login Flow
Setup	Batch-provision existing users	Batch Import Contacts
Operations	Call the identity synchronization API during account creation	Contact Identity Synchronization and Token Updates
Operations	Update credentials at login and token refresh	When to Call
Operations	Revoke credentials at logout	Logout and Credential Revocation
Advanced	Synchronize personalized attributes such as department and membership level	Synchronize Contact Data
Advanced	Use query metadata to limit each user's knowledge base retrieval scope	Knowledge Management Permissions (Query Metadata) Overview

Web Chat Embedding and SDK

Add a snippet of embed code to your web page to put a MaiAgent AI Agent on your website. Once the embed code loads `embed.min.js` (hereafter "the SDK"), the SDK creates a chat button and an iframe chat window on the page, and exposes a global `MaiAgent` JavaScript API for programmatic control.

This page is the complete technical reference for Web Chat embedding. Related topics:

[Integrating MaiGPT: Choose the Right Assistant and Add Capabilities](#) — the MaiGPT product vs. MaiGPT window mode, and how to enable embedded assistants to generate images and use tools

[Web Chat MaiGPT Mode Embedding](#) — a ChatGPT-like, full-featured conversation interface

[WebChat Page Context Injection](#) — inject page information into the LLM System Prompt

[Contact Identity Sync and Token Refresh](#) — create contacts and credentials from the backend

[Software Vendor Integration Guide](#) — the complete journey of embedding plus permission integration (batch backfill, login sync, logout revocation)

1. Quick Start

Add the following before `</body>` on your page:

```
<script>
  window.maiagentChatbotConfig = {
    webChatId: 'your-web-chat-id',
    baseUrl: 'https://chat.maiagent.ai/web-chats',
    primaryColor: '#1890ff',
  }
</script>
<script src="https://chat.maiagent.ai/js/embed.min.js" defer></script>
```

Get `webChatId` from the Web Chat settings page in the admin console (see [Connecting Chat Platforms: Website](#); the embed code generated by the console fills it in automatically)

The `baseUrl` and SDK URL above use the SaaS environment (`chat.maiagent.ai`) as an example; for private cloud on-premises deployments, replace them with your environment's domain

`window.maiagentChatbotConfig` **must be defined before the SDK script loads**

Default behavior: a chat button appears at the bottom-right of the page. Clicking it opens a floating chat window, and the user can switch between the floating and sidebar window modes.

Initialization Timing

After loading, the SDK registers a `document.body.onload` handler to run initialization; normally no manual intervention is needed. When you need to control the initialization timing (for example, waiting for an SPA container to mount, or deferring rendering):

Lazy-load the script: dynamically insert the `embed.min.js`